AWS KMS — Fundamentals Notes

Encrypt Data with AWS KMS | Simple Solutions Architect Revision Guide

How to use this document

Use this document when you forget the fundamentals. Each concept is explained in simple words, followed by its purpose, why it is used, and a simple example.

1. What problem does AWS KMS solve?

Companies store sensitive information in databases, files, and applications. If stored data is exposed, encryption helps prevent unauthorized people from understanding it.

Encryption

Simple meaning: Encryption changes readable data into unreadable ciphertext using an encryption key.

Purpose: Protect data from unauthorized access.

Why use it: Someone who obtains encrypted data should not automatically be able to read it.

Example: A database stores customer information. Encryption protects the stored information.

Plaintext

Simple meaning: The original readable data before encryption.

Purpose: Represent the normal form of data.

Why use it: You need to understand what is being protected before understanding encryption.

Example: Customer name: Ahmed.

Ciphertext

Simple meaning: The unreadable form of data produced by encryption.

Purpose: Represent protected data.

Why use it: It prevents the stored information from being directly understood without decryption.

Example: The original name is transformed into an unreadable encrypted value.

Encryption Key

Simple meaning: Information used by an encryption algorithm to encrypt and decrypt data.

Purpose: Provide the cryptographic control needed to protect data.

Why use it: The correct key is required for authorized decryption.

Example: Think of it like a digital key used to lock and unlock protected data.

2. What is AWS KMS?

AWS Key Management Service (KMS)

Simple meaning: A managed AWS service used to create, manage, and control encryption keys.

Purpose: Manage encryption keys used to protect AWS resources.

Why use it: Keys need controlled access, management, and lifecycle handling.

Example: Instead of an application managing its own encryption key, it can use a KMS key.

Key Management System

Simple meaning: A system that securely manages encryption keys and controls who can use them.

Purpose: Keep encryption keys organized and protected.

Why use it: Encryption keys are security-sensitive and should not be handled without controls.

Example: A company manages its encryption keys centrally through KMS.

3. Symmetric vs. Asymmetric Encryption

Symmetric Encryption

Simple meaning: The same key is used to encrypt and decrypt data.

Purpose: Efficiently protect data.

Why use it: It is generally faster and suitable for large amounts of data.

Example: The KMS key used in this project is symmetric because DynamoDB data needs efficient encryption.

Asymmetric Encryption

Simple meaning: A pair of keys is used: a public key and a private key.

Purpose: Support use cases where public/private key cryptography is needed.

Why use it: The two keys have different roles.

Example: A public key can be shared while the private key remains protected.

Remember: Symmetric = one key. Asymmetric = public key + private key.

4. Types of KMS Keys Used in DynamoDB

Option	Who manages it?	Main idea
AWS owned key	AWS	Simple encryption with no customer visibility or control over the key.
AWS managed key	AWS KMS / AWS	KMS manages the key for an AWS service.
Customer managed key	Customer	Customer creates and manages the KMS key and has more control.

Customer Managed Key

Simple meaning: A KMS key that you create and manage.

Purpose: Give yourself more control over the encryption key.

Why use it: You can manage its policy, users, and lifecycle.

Example: The project used nextwork-kms-key to protect the DynamoDB table.

5. KMS Key Usage

Encrypt and Decrypt

Simple meaning: A key configured for encryption can protect data and decrypt data that was encrypted with the key.

Purpose: Define what cryptographic purpose the key supports.

Why use it: The key must support the operations required by the workload.

Example: The project selected Encrypt and decrypt for the DynamoDB encryption key.

Key Administrator

Simple meaning: A person or role that manages the KMS key, its policy, and lifecycle.

Purpose: Manage the key.

Why use it: Key management can be separated from normal key usage.

Example: The IAM Admin User was selected as the key administrator.

Key User

Simple meaning: A person or role allowed to use the key for cryptographic operations.

Purpose: Allow approved identities to use the key.

Why use it: A user may need to decrypt data without being allowed to manage the key itself.

Example: The test user was later added as a key user.

Key Lifecycle

Simple meaning: The stages a key goes through, such as creation, use, and deletion.

Purpose: Manage the key safely throughout its existence.

Why use it: Incorrect key deletion can affect data that depends on the key.

Example: After the lab, the KMS key was scheduled for deletion.

6. What is a KMS Key Policy?

KMS Key Policy

Simple meaning: A policy attached directly to a KMS key that defines who can access the key and what they can do.

Purpose: Control usage of the encryption key.

Why use it: KMS key access is a separate security decision from database access.

Example: The test user was added to the key policy so it could use the key.

IAM Policy

Simple meaning: A policy that grants permissions to IAM users, groups, or roles.

Purpose: Control what an AWS identity can do.

Why use it: Users should receive only the permissions needed for their tasks.

Example: The test user received AmazonDynamoDBFullAccess but initially did not have KMS key access.

> **Important: IAM access to DynamoDB does not automatically mean permission to use a customer managed KMS key.**

7. Amazon DynamoDB

DynamoDB

Simple meaning: An AWS managed NoSQL database service for storing and retrieving data.

Purpose: Provide fast and flexible database storage.

Why use it: Applications need a managed service to store application data.

Example: The project created nextwork-kms-table and stored an item in it.

Partition Key

Simple meaning: An attribute DynamoDB uses to organize and distribute items.

Purpose: Help DynamoDB organize data and efficiently access it.

Why use it: It is a fundamental part of how DynamoDB stores and retrieves items.

Example: The project used id as the partition key.

8. Encryption at Rest

Data at Rest

Simple meaning: Data that is stored and not currently being moved or actively used.

Purpose: Describe stored data that needs protection.

Why use it: Databases, files, and backups can contain sensitive information while stored.

Example: The DynamoDB table contains data at rest.

Encryption at Rest

Simple meaning: Encryption applied to stored data.

Purpose: Protect stored data.

Why use it: It adds a security layer to information sitting in storage.

Example: The DynamoDB table was configured to use the customer managed KMS key for encryption at rest.

> **Remember the three states: At rest = stored data. In transit = data moving. In use = data actively being processed.**

9. How does data flow?

Step	What happens
1	User or application requests data from DynamoDB.
2	DynamoDB retrieves the stored encrypted data.
3	The request must be authorized to use the KMS key.
4	For an authorized request, the protected data is decrypted.
5	DynamoDB returns usable data to the authorized user or application.

Transparent Data Encryption

Simple meaning: Stored data remains encrypted, while authorized access can receive the data in usable form.

Purpose: Protect data at rest without requiring the application to manually decrypt every database item.

Why use it: Authorized users can work with the data normally while the stored data remains encrypted.

Example: You could see the DynamoDB item because your identity was authorized to use the KMS key.

10. How is it secured?

IAM Access Control

Simple meaning: IAM determines what an AWS identity is allowed to do.

Purpose: Control access to AWS resources and services.

Why use it: It prevents users from receiving unnecessary permissions.

Example: The test user was given DynamoDB permissions.

Least Privilege

Simple meaning: Give an identity only the permissions required for its job.

Purpose: Reduce unnecessary access.

Why use it: If an identity is compromised, limited permissions reduce potential impact.

Example: The test user intentionally received DynamoDB access without KMS access.

KMS Permission

Simple meaning: Permission to perform a specific cryptographic operation with a KMS key.

Purpose: Control who can encrypt or decrypt using the key.

Why use it: A user may have database access but still need KMS permission to access protected data.

Example: The test user initially lacked permission to decrypt.

11. What usually breaks?

Problem	What it usually means	First thing to check
Access Denied	A required permission is missing.	Read the exact denied action.
kms:Decrypt denied	The identity cannot decrypt using the KMS key.	Check KMS key policy and applicable IAM permissions.
DynamoDB works but encrypted data does not	Database access exists, but KMS access is missing.	Check the KMS key permissions.
Wrong encryption key	The table may be using a different KMS key.	Check DynamoDB encryption settings.
User cannot use the key	The identity is not allowed for the required operation.	Review key users and key policy.

Simple troubleshooting method

1. Read the exact error message.
2. Identify the AWS action that was denied.
3. Check whether the user has the required service permission.
4. If encrypted data is involved, check KMS permissions.
5. Check the KMS key policy.
6. Add only the required permission.
7. Retry and verify.

12. The most important lesson from this project

The project deliberately created a test user with AmazonDynamoDBFullAccess but without permission to use the KMS key. The user could reach the DynamoDB table but received an Access Denied error when trying to view the encrypted items. The administrator then added the user to the KMS key policy. After that, the user could access the items.

> **Memory rule: Resource access and encryption-key access are separate security layers.**

13. Simple real-world analogy

Imagine a locked room containing a locked safe.

IAM = decides who can enter the room.
DynamoDB = the room containing the data.

Encryption = the lock protecting the data.
KMS = manages the encryption key.
KMS key policy = decides who can use that key.

A person may be allowed into the room but still be unable to open the safe. That is the main security lesson demonstrated by this project.

14. Quick Revision Sheet

- KMS → manages encryption keys.
- Encryption → plaintext becomes ciphertext.
- Decryption → ciphertext becomes readable data.
- Symmetric → same key encrypts and decrypts.
- Asymmetric → public key + private key.
- Customer managed key → customer has more control.
- Key administrator → manages the key.
- Key user → uses the key.
- Key policy → controls use of the KMS key.
- IAM policy → controls AWS identity permissions.
- DynamoDB → managed NoSQL database.
- Encryption at rest → protects stored data.
- Transparent encryption → stored data is encrypted but authorized access can receive usable data.
- DynamoDB permission alone does not automatically provide KMS key permission.
- kms:Decrypt → allows an authorized principal to decrypt with the key.

Source: NextWork project text supplied by the user: “Encrypt Data with AWS KMS.” The source covers KMS, encryption, DynamoDB, key policies, IAM permissions, transparent data encryption, and the test-user access scenario. ■filecite■turn1file0■L41-L67■